

Digital Professor Tutoring Prototype

Scoping Plan and System Architecture

Digital Professor Team

September 28, 2026

1 Project Pivot and Goal

The Digital Professor prototype will focus on an **assessment-based tutoring interaction** rather than attempting to implement a broad multimodal tutoring platform immediately.

The central question is:

How can an AI replicate the steps taken by a human tutor in a generalizable way?

The prototype should mimic the flow of a tutoring session in which a student arrives with a question or attempted solution. Rather than immediately giving an answer, the Digital Professor should determine what the student has tried, diagnose where the difficulty lies, choose an appropriate intervention, return control of the problem to the student, and evaluate whether the student actually understands the concept.

Two human tutoring workflows were used as initial examples:

- a statistics tutoring workflow involving sufficiency and the Neyman–Fisher factorization theorem;
- a calculus tutoring workflow involving substitution and integration by parts.

Although the mathematical content differs, both workflows exhibit a common tutoring structure. The goal is to abstract this structure so that the Digital Professor is not tied to a particular problem, course, or mathematical method.

2 Generalized Tutoring Loop

The tutoring workflows suggest that the Digital Professor should operate as a repeated decision loop rather than a simple question–answer system.

1. **Elicit.** Ask the student what they have tried, what they currently think, and where they became stuck.
2. **Diagnose.** Determine the student’s current approach, which parts are correct, and whether there is an execution error, conceptual misconception, or missing prerequisite.
3. **Select an intervention.** Choose the smallest useful pedagogical action rather than automatically providing the complete solution.
4. **Return control to the student.** Ask the student to attempt the next step, revise the solution, or explain their reasoning.

5. **Evaluate the response.** Determine whether the new response provides evidence of progress and whether the original difficulty has been resolved.
6. **Adapt or remediate.** If the student remains stuck, change the intervention. This may involve a more explicit hint, a simpler example, comparison with another problem, a prerequisite review, or direct explanation.
7. **Check understanding.** Once the student reaches a correct step or answer, ask them to explain why the method works rather than treating correctness alone as evidence of understanding.
8. **Test transfer.** Give a similar and/or contrasting problem to determine whether the student can recognize when the concept should and should not be applied.

This loop continues until there is sufficient evidence that the student can both perform the task and explain or transfer the relevant concept.

3 Pedagogical Action Vocabulary

The two tutoring workflows suggest a relatively small set of recurring tutor actions. These can form a general action vocabulary for the Digital Professor.

Action	Purpose	Example DP Behavior
Elicit	Expose the student’s current thinking.	“Show me what you tried and where you became unsure.”
Probe	Test prerequisite or conceptual knowledge.	“What theorem or technique do you think applies here?”
Inspect	Obtain intermediate reasoning rather than only a final answer.	“Walk me through how you got from this step to the next.”
Diagnose	Identify the likely error or misconception.	Determine whether the approach is appropriate and where reasoning diverges.
Hint	Provide directional help without solving the problem.	Ask a question that points toward the next useful observation.
Contrast	Distinguish similar-looking concepts or methods.	Compare the current problem with an easier example and ask what differs.
Explain	Repair a misconception directly when questioning is insufficient.	Explain why the proposed method does not apply in the current situation.
Demonstrate	Use a simplified example or representation.	Introduce a small numerical example that exposes the relevant structure.
Retry	Return responsibility for solving to the student.	“Now try that step again using what we just discussed.”
Verify	Evaluate a revised step or explanation.	Check whether the correction resolves the earlier error.
Check understanding	Determine whether correctness reflects conceptual understanding.	“Why does this method work here?”
Transfer	Test generalization beyond the current problem.	Give one similar problem and one contrasting problem.

The system should therefore make two related decisions:

1. What does the student currently understand?
2. Which pedagogical action should occur next?

The wording of the response is secondary to selecting an appropriate tutoring action.

4 Student State

A major requirement is maintaining a representation of the student's current state. This is different from simply storing the conversation history.

A conceptual student-state record could contain:

- current problem or learning objective;
- student's current approach;
- current solution step;
- student's stated reasoning;
- components that appear correct;
- suspected misconception or error;
- potentially missing prerequisite knowledge;
- interventions already attempted;
- student's response to those interventions;
- evidence of conceptual understanding;
- evidence of transfer;
- uncertainty in the system's diagnosis.

For example, a student might say that they attempted to derive the distribution of a statistic in order to prove sufficiency. The conversation history stores the student's statement. The student state should instead capture that the student has identified a relevant statistic but may not yet understand the criterion required to establish sufficiency.

This structured state allows the Digital Professor to reason about *what to do next* rather than repeatedly reconstructing the tutoring situation from raw dialogue.

5 Evidence of Understanding

A correct final answer should not automatically terminate a tutoring interaction. Both human tutoring workflows continue beyond correctness by asking for explanations or additional examples.

A useful conceptual progression is:

Level	Observed Evidence
0	No evidence of relevant understanding
1	Recognizes the relevant concept or approach
2	Executes the approach with assistance
3	Executes the approach independently
4	Explains why the approach works
5	Transfers the concept to a new or contrasting problem

These levels are not intended to be a final grading scale. They provide a framework for determining what evidence the Digital Professor should seek before concluding that a tutoring objective has been achieved.

6 System Architecture

The tutoring loop should be treated as the central product logic. Models, retrieval, syllabus parsing, interaction modalities, and the avatar support this loop rather than define it.

7 Team Integration

Each workstream should ultimately provide information that can be consumed by the tutoring loop.

7.1 Interaction Methods — Jonathan

The interaction layer determines what evidence can be collected from the student and how tutoring interventions are communicated.

Relevant student inputs include:

- question;
- attempted solution;
- intermediate steps;
- verbal or written explanation;
- statement of uncertainty;
- response to a hint;
- revised solution;
- answer to a comprehension question;
- transfer-problem attempt;
- attached image, document, handwriting, equation, or other artifact.

Relevant Digital Professor outputs include:

- elicitation question;
- diagnostic probe;
- hint;
- explanation;
- comparison or example;
- correction;
- retry request;
- comprehension check;
- transfer problem.

The interaction-method research should therefore focus on which interaction methods best capture useful *evidence of student understanding*, not simply which modalities are technically available.

7.2 Syllabus Generation and Parsing — Ovunc

The syllabus and course-processing component should provide structured course context to the tutoring system.

Potential outputs include:

- course and topic;
- learning objectives;

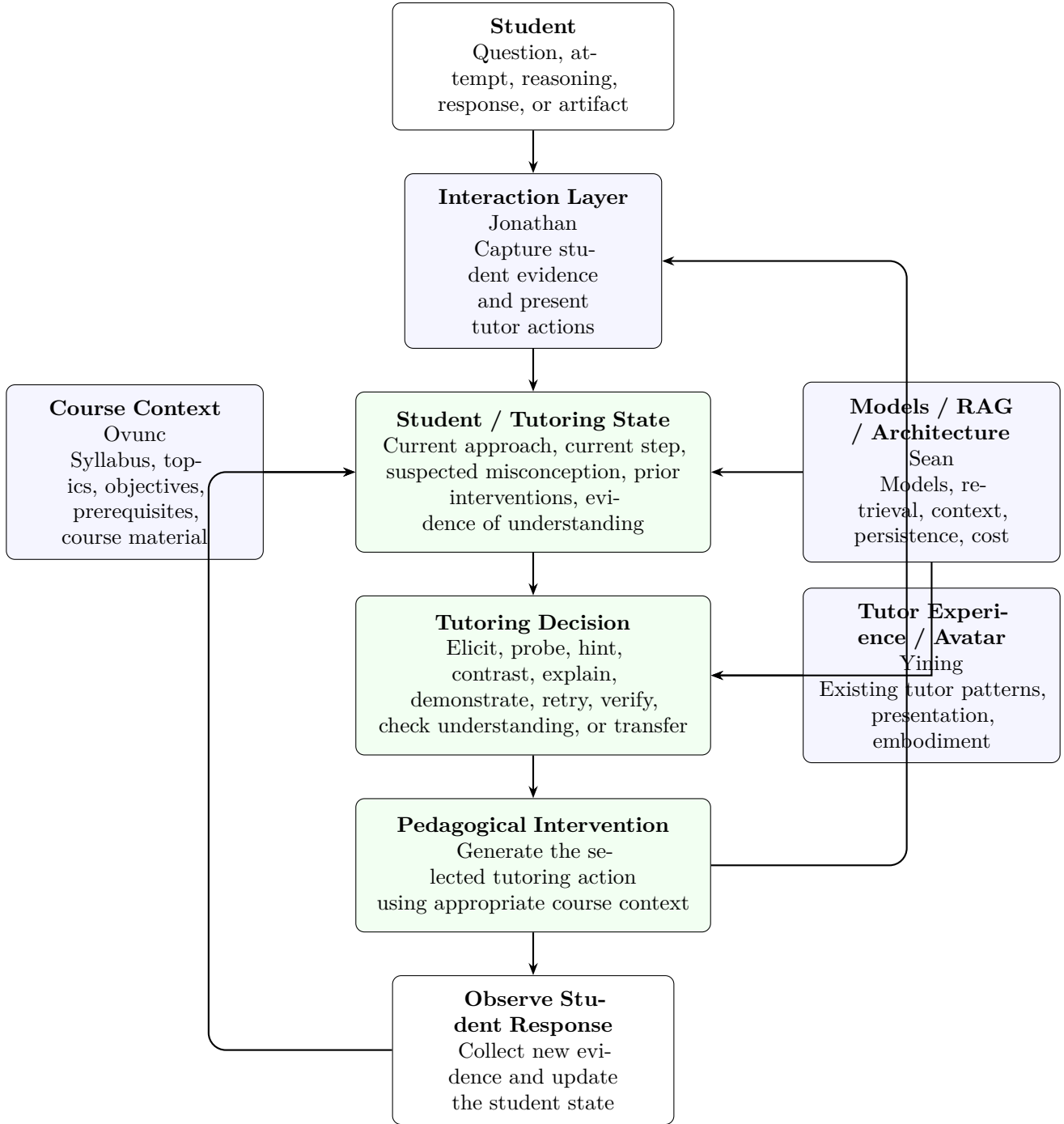


Figure 1: Conceptual architecture for the Digital Professor tutoring loop. The system repeatedly gathers student evidence, updates its estimate of the student’s state, selects a pedagogical action, and observes the result.

- prerequisite concepts;
- course terminology;
- relevant source material;
- assessment context;
- potentially relevant examples or practice problems.

This information provides the reference frame against which the student’s reasoning can be interpreted.

7.3 Existing AI Tutors and Avatar — Yining

This workstream should identify how existing tutoring systems implement pedagogical interactions and how the Digital Professor should present those interactions to students.

Relevant outputs include:

- common tutoring patterns used by existing systems;
- approaches to hints and scaffolding;
- handling of incorrect answers;
- approaches to comprehension checking;
- interaction patterns worth reproducing or avoiding;
- avatar and presentation options;
- possible role of embodiment in the tutoring experience.

7.4 Models, RAG, Cost, and Architecture — Sean

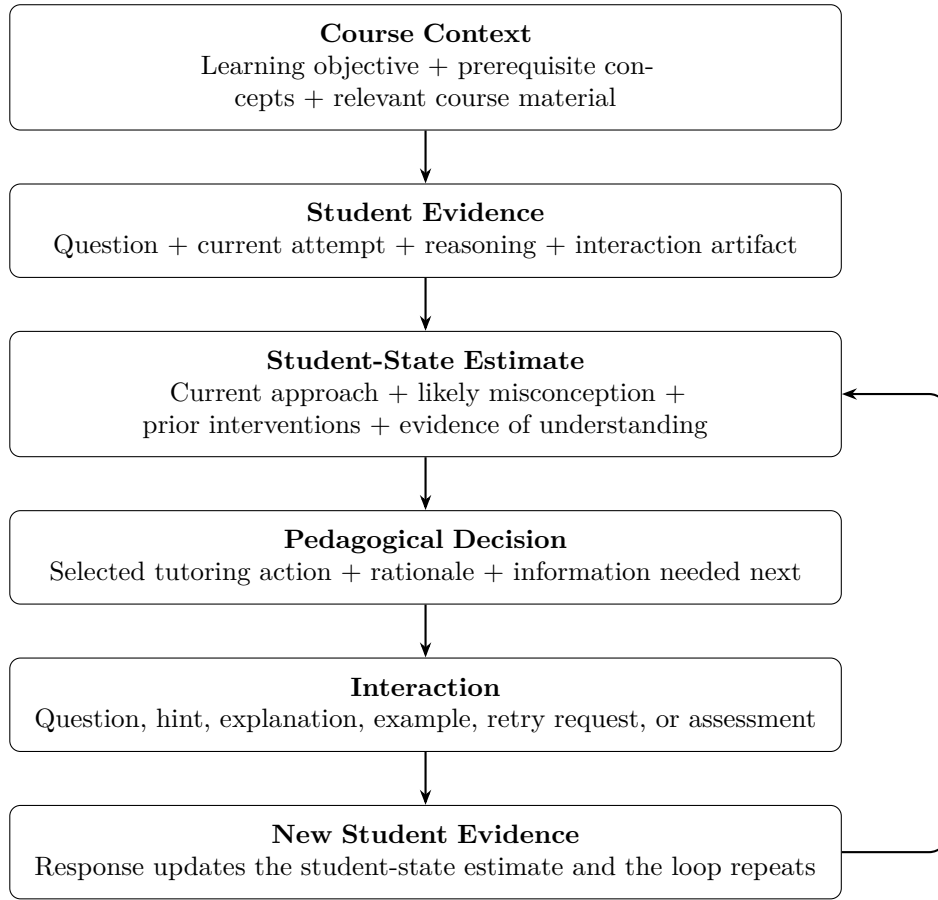
This workstream determines how the required tutoring operations could be implemented technically.

Relevant questions include:

- Which models can reliably interpret student reasoning?
- What context should be passed to the model?
- What information should persist between tutoring turns?
- How should course material be retrieved?
- Should diagnosis and response generation use the same model?
- Is an independent verifier useful?
- Which operations require RAG?
- What is the expected cost of a tutoring session?

8 Information Flow Between Components

A useful integration principle is that each component should produce a structured artifact that can be consumed by another component.



This interface allows individual project components to change without changing the conceptual tutoring process.

9 Evaluation Plan

Evaluation should test whether the Digital Professor reproduces useful *tutoring behavior*, not merely whether it can solve the underlying problem.

The two human tutoring workflows provide initial reference trajectories. Additional workflows should be collected across different tutors, subjects, student mistakes, and levels of understanding.

9.1 Evaluation Unit

Each tutoring turn can be represented as:

Student state $\rightarrow$ student evidence $\rightarrow$ tutor decision $\rightarrow$ intervention $\rightarrow$ new student evidence.

Evaluation can therefore examine each stage separately.

9.2 1. Diagnosis Quality

Given the student's work and course context, evaluate whether the system correctly identifies:

- the student’s current approach;
- which steps are correct;
- the location of the first meaningful error;
- the likely misconception;
- missing prerequisite knowledge;
- whether there is enough evidence to make a diagnosis.

A strong system should be able to report uncertainty rather than inventing a misconception when the available evidence is insufficient.

9.3 2. Pedagogical Decision Quality

Given a student state, evaluate whether the system selects an appropriate next tutoring action.

Example questions include:

- Did the system ask for more information when necessary?
- Did it provide a hint when a full explanation was unnecessary?
- Did it avoid giving away the answer too early?
- Did it recognize when direct explanation was appropriate?
- Did it return control of the problem to the student?
- Did it check understanding after apparent success?
- Did it test transfer when appropriate?

Human tutors can independently label what they would do next and compare those decisions with the Digital Professor.

9.4 3. Intervention Quality

Even if the correct pedagogical action is selected, the generated intervention must still be evaluated.

Possible criteria include:

- mathematical or domain correctness;
- relevance to the student’s actual difficulty;
- clarity;
- appropriate level of detail;
- consistency with course material;
- usefulness of the hint or explanation;
- whether the intervention preserves student agency;
- whether the response introduces unnecessary information.

This separates the question “Did DP choose the right tutoring strategy?” from “Did DP execute that strategy well?”

9.5 4. Student-State Update Quality

After receiving a student response, evaluate whether the system correctly updates its estimate of understanding.

For example, a correct numerical answer should not necessarily be interpreted as conceptual mastery. The system should distinguish between:

- guessing;
- solving with substantial assistance;
- independent execution;
- correct conceptual explanation;
- successful transfer.

9.6 5. End-to-End Tutoring Quality

Full tutoring sessions can then be evaluated on whether the system:

- reaches the correct diagnosis efficiently;
- provides appropriate scaffolding;
- adapts when the first intervention fails;
- avoids unnecessary solution disclosure;
- correctly identifies when the student understands;
- produces evidence of transfer;
- remains grounded in the intended course context.

The human tutoring workflows can initially act as reference cases, but the Digital Professor should not be required to reproduce the exact same sequence. Different tutoring paths may be equally valid if they lead to appropriate diagnosis, scaffolding, and demonstrated understanding.

10 Proposed Assessment-Based Prototype

Before implementing a full application, the team can evaluate the concept as a set of tutoring scenarios.

For each scenario, provide the system with:

1. a learning objective;
2. relevant course context;
3. a problem;
4. a simulated or recorded student attempt;
5. the student's explanation of their reasoning;
6. optional follow-up responses.

At each stage, ask the Digital Professor to produce a structured assessment:

- current interpretation of the student's approach;
- what appears correct;
- suspected error or misconception;
- confidence in that diagnosis;
- next pedagogical action;
- actual tutoring response;
- what evidence it expects from the student next.

The student's next response is then revealed and the process repeats.

This creates an assessment framework for the tutoring architecture without requiring the team to build the complete application first.